



U5 - CI/CD

2º DAW

2025/2026

Juanra Collado – jr.colladoesteve@edu.gva.es

CI/CD. ¿Qué es?

- CI/CD son las siglas de **Integración Continua** (*Continuous Integration*) y **Entrega/Despliegue Continuo** (*Continuous Delivery/Deployment*).
- Podemos decir que CI/CD es un conjunto de prácticas que permiten a los equipos de desarrollo de software **automatizar tareas** como la integración del código, realización de tests y el despliegue de las aplicaciones de un manera rápida y segura.

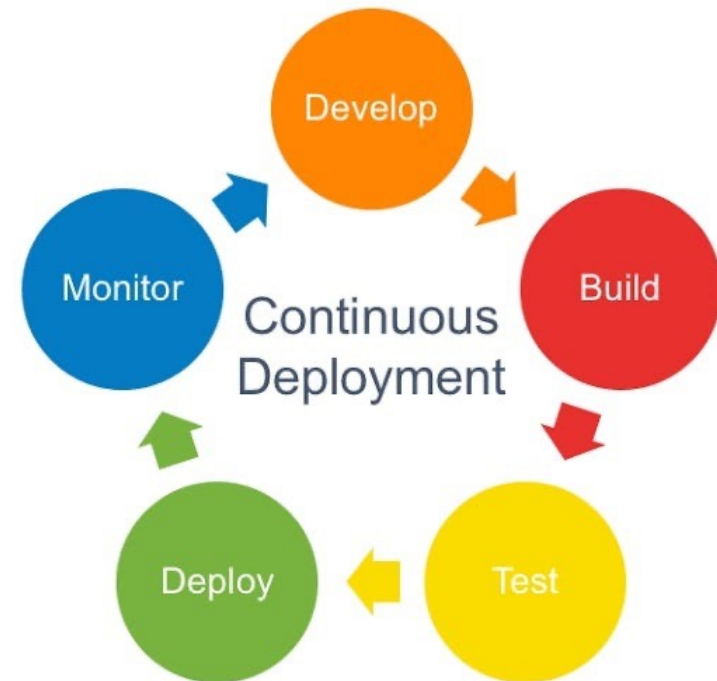
CI

- Automatiza comprobaciones del código cuando alguien hace un push.
- En esta unidad **solo veremos la idea**, no la implementación ya que depende de test unitarios y no son objeto de estudio de este módulo.



CD

- Automatiza la actualización del servidor cada vez que hay un push válido.
- Evita FTP/SFTP/SSH manual.
- Garantiza que la web está siempre actualizada.



Herramientas CI/CD más utilizadas

- GitHub Actions

- La trabajaremos en clase
- Fácil de usar
- Ideal para proyectos pequeños/medianos
- Estándar si se usa GitHub.



GitHub Actions

- Jenkins

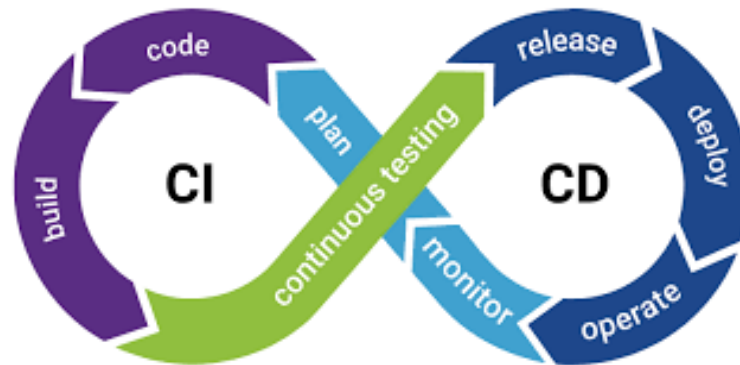
- Software independiente, muy usado en empresas grandes
- Se instala en un servidor propio
- Altamente configurable
- Muy potente pero más complejo
- Integrable con cualquier tecnología y tipo de repositorio
- Requiere mantenimiento
- Soportado por miles de plugins



Jenkins

CD vs CI

- CI = Comprueba si el proyecto **funciona**
- CD = **Despliega** el proyecto automáticamente



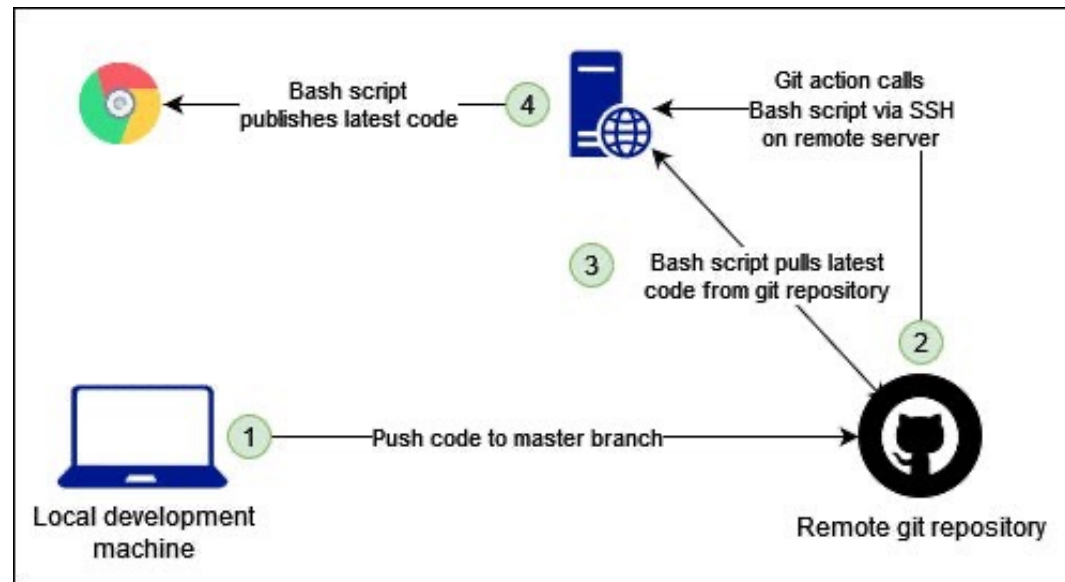
¿Qué se automatiza exactamente?

- Por ejemplo, para una app desplegada con Sail:
 - Actualizar código con git pull
 - Apagar Sail (docker)
 - Reconstruir contenedores
 - Levantar la aplicación actualizada
 - Todo esto **automáticamente** al hacer un push a main.
 - El despliegue automático está pensado para entornos de producción por lo que no se suele usar en local.
 - Sin embargo, para pruebas es común querer utilizarlo en local. Más adelante veremos las particularidades de este entorno para CD.
-

¿Cómo funciona un CD real en un servidor?

- En un entorno profesional, el despliegue automático funciona así:
 - El desarrollador hace **push a la rama main**
 - GitHub Actions detecta el cambio
 - GitHub Actions **se conecta al servidor por SSH**
 - El servidor ejecuta un **script de despliegue**
 - GitHub no despliega, ordena al servidor que ejecute unos comandos que despliegan.
 - La aplicación se actualiza automáticamente y pasa a tener los cambios que se han subido al main

¿Cómo funciona un CD real en un servidor?



¿Qué necesitamos en el servidor para que esto funcione?

- Una IP (o nombre de dominio)
- Un usuario
- Servicio SSH
- Apps necesarias (Docker/componentes)
- El proyecto ya corriendo en la máquina (o su código)
- Permisos adecuados para ejecutar comandos.



¿Qué es el script de despliegue?

- Conjunto de comandos que el servidor ejecuta para actualizar la aplicación que ya está desplegada.
 - Dependiendo del lenguaje, del entorno y de otras variables, los comandos que se ejecutan variarán.
 - En esta unidad estudiaremos como desplegar aplicaciones Laravel utilizando Sail y mediante componentes.
-

Ventajas del despliegue automático

- No hay despliegues manuales
 - El servidor siempre está sincronizado con GitHub
 - Se reduce el riesgo de errores
 - Se puede desplegar muchas veces al día sin miedos ni problemas
 - Es el estándar en desarrollo profesional
-

CD en local

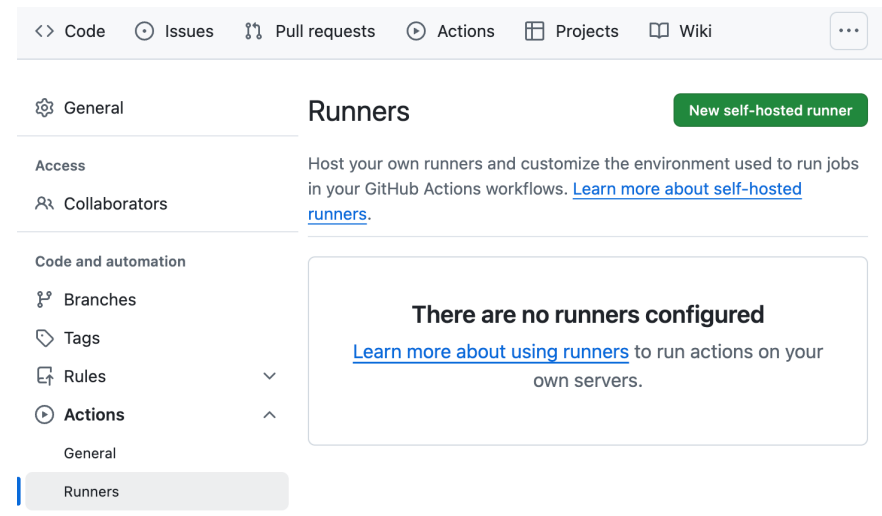
- Cuando queremos probar o utilizar el despliegue en local nos encontramos con algunos problemas que chocan conceptualmente con el despliegue automático.
 - Hemos dicho, que parte del proceso es que GitHub se conecte por SSH a nuestro servidor.
 - Si estamos en local, este punto no podrá hacerlo. Para ello:
 - Self-Hosted runner
 - Todo lo que viene a continuación, solo hay que realizarlo si el despliegue es en local, **NUNCA** si es en remoto (VPS, AWS, etc)
-

CD en local. ¿Qué es un self-hosted runner?

- Es la solución para el problema de conexión por SSH desde GitHub a nuestro PC.
 - GitHub no puede llegar a nuestro localhost, por tanto hacemos lo contrario.
 - Nuestro PC se conecta a GitHub y se ofrece para ejecutar los workflows necesarios.
 - El resto de proceso, se realizará de la misma forma (script, reconstrucción de contenedores, etc).
-

Práctica 1 - CD en local

- El primer paso para configurar CD en local, es configurar el runner en GitHub. Para ello vamos al repositorio que queremos desplegar y *Settings->Actions->Runners*.
- Pulsamos *New self-hosted runner*



Práctica 1 - CD en local

- Seleccionamos el SO que utilizamos y nos dará una serie de instrucciones que hemos de seguir para instalar nuestro self-hosted runner.
 - El runner puede estar instalado en cualquier carpeta del SO. Elige una ubicación donde no moleste y donde no lo elimines por equivocación.
 - Es importante que las instrucciones se **copien y se peguen** ya que hay referencias al repositorio en estas instrucciones.
 - Cuando se entra en el *wizard* de la configuración (en el config.sh), podemos dejarlo todo por defecto
-

Práctica 1 - CD en local

```
juanra@MacBook-Air-de-Juanra actions-runner % ./config.sh --url https://github.com/JuanraCollado/juanracollado.github.io --token BBWBVT2JVLTD2TSY4FXVFSLJHUNZA
```



```
# Authentication
```

```
✓ Connected to GitHub
```

```
# Runner Registration
```

```
Enter the name of the runner group to add this runner to: [press Enter for Default]
```

```
Enter the name of runner: [press Enter for MacBook-Air-de-Juanra]
```

```
[This runner will have the following labels: 'self-hosted', 'macOS', 'X64']
```

```
Enter any additional labels (ex. label-1,label-2): [press Enter to skip]
```

```
✓ Runner successfully added
```

```
# Runner settings
```

```
[
```

```
Enter name of work folder: [press Enter for _work]
```

```
✓ Settings Saved.
```

Práctica 1 - CD en local

- Una vez configurado, sólo falta lanzarlo y veremos como automáticamente ya está preparado para recibir trabajos (escuchando de GitHub).
 - Si cerramos la terminal, el runner deja de estar funcionando.

```
[juanra@MacBook-Air-de-Juanra actions-runner % ./run.sh
```

```
✓ Connected to GitHub
```

```
Current runner version: '2.329.0'
```

```
2025-12-13 06:59:39Z: Listening for Jobs
```

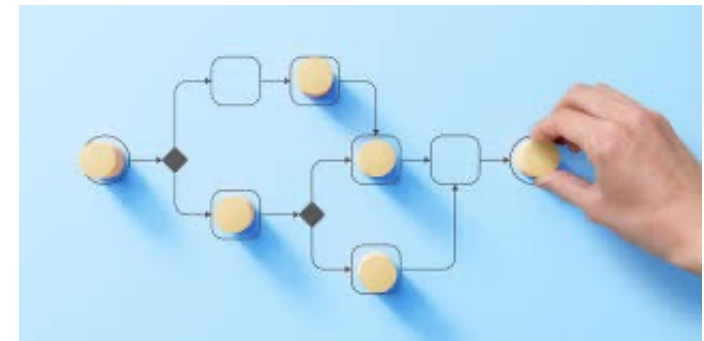


CD en local y remoto

- Una vez tenemos nuestro runner configurado en local, **el resto de configuración es común tanto si estamos en local o en remoto.**
 - Recordatorio. Para CD siempre necesitamos, independientemente del entorno.
 - Repositorio Github 
 - Hemos de asegurarnos que nuestro proyecto esté subido a GitHub.
 - Workflow de Github Actions. 
 - Script de despliegue 
 - Máquina donde se realiza el despliegue con todo preparado y web funcionando 
 - La máquina donde desplegamos ha de estar preparada (seguridad, accesible, runner si es en local) y ha de tener instalados los componentes necesarios dependiendo del tipo de despliegue.
 - Por ejemplo docker/docker compose si el despliegue es con sail o el resto de componentes (php, composer, etc) si es por componentes.
 - Tenéis en Aules guías para realizar despliegues tanto con Sail como por componentes.
 - **Antes de incluir CD, es necesario que nuestra web funcione sin CD.**
 - Lo único que cambia entre local y remoto es la máquina donde se ejecuta el script y cómo llega el workflow a esta máquina.
 - Al remoto llega por SSH y al local gracias al runner (instalado en la P1)
-

¿Qué es un workflow de GitHub Actions?

- Un **workflow** es un archivo que indica a GitHub Actions:
 - **Cuándo** debe ejecutarse (evento)
 - **Dónde** debe ejecutarse (SSH o runner)
 - **Qué comandos** debe ejecutar (lista de steps)
- Los workflows se escriben en formato **YAML** y forman parte del repositorio.



¿Dónde se guardan los workflows?

- GitHub Actions sólo detecta workflows si están en la siguiente ruta dentro del proyecto:
 - *.github/workflows/*
 - El workflow que utilizaremos para CD es siempre el mismo y es bastante sencillo, cambiará solamente la parte del script (comandos)
 - GitHub Actions tiene potencia para realizar workflows y acciones mucho más complejas fuera del alcance de este módulo.
-

Workflow

- Un workflow típico tiene las siguientes partes:
 - name → nombre descriptivo.
 - En nuestro caso lo llamaremos *Deploy Laravel Sail*
 - on → evento que lo activa
 - En nuestro caso será cuando se realiza un *push* sobre la rama *main*
 - jobs → trabajos a ejecutar.
 - En nuestro caso el job es simple, *deploy*. Cada job, tiene *runs-on* y *steps*

Workflow

- runs-on → en qué máquina se ejecuta el workflow
 - Si estamos en local-> self-hosted
 - Si estamos en remoto-> ubuntu-latest
 - Esta es la máquina de GitHub. Siempre es así.
- steps → cada job tiene una lista de acciones.
 - El despliegue (script de despliegue) será una de esta lista de acciones.

Workflow

- Recuerda que el fichero se encuentra en `.github/workflows/deploy.yml`
 - Podría ponerle otro nombre como `deploy-laravel.yml` o similar.
- A este fichero de workflow falta incluirle el script (serie de instrucciones) de despliegue.
 - Se añadirá como un *step* más en las siguientes diapositivas

```
name: Deploy Laravel Sail

on:
  push:
    branches: [ "main" ]

jobs:
  deploy:
    # Opción A – Local
    runs-on: self-hosted

    # Opción B – Remoto (VPS)
    # runs-on: ubuntu-latest

    steps:
```


CD en local y remoto

- Vamos a ver el estado de nuestro despliegue:
- Para CD siempre necesitamos, independientemente del entorno.
 - Repositorio Github 
 - Workflow de Github Actions. 
 - Script de despliegue 
 - Máquina donde se realiza el despliegue con todo preparado y web funcionando 

Script de despliegue

- Las acciones a realizar por el script son las mismas tanto en local como en remoto, pero cambia como nos conectamos a él.
- El script no es un archivo aparte, sino que se incluyen las instrucciones dentro de un *step* del workflow
- El script no es más que las instrucciones que se ejecutarán al hacer el push. Pueden cambiar dependiendo del entorno, lenguaje, etc

Script de despliegue

- Para una aplicación Laravel desplegada con Sail sería el siguiente script:

```
cd /ruta/al/proyecto
git pull
./vendor/bin/sail down
./vendor/bin/sail up -d --build
./vendor/bin/sail artisan migrate --force
```

- Para una aplicación Laravel desplegada por componentes sería el siguiente script:

```
cd /ruta/al/proyecto
git pull
composer install --no-dev --optimize-autoloader
php artisan optimize:clear
php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan storage:link || true
sudo systemctl reload php8.3-fpm
sudo systemctl reload nginx
```

Script de despliegue

- Para local:

```
steps:
- name: Deploy (Local) con Sail
  shell: bash
  run: |
    set -e
    cd /ruta/al/proyecto
    git pull
    ./vendor/bin/sail down
    ./vendor/bin/sail up -d --build
    ./vendor/bin/sail artisan migrate --force
```

```
steps:
- name: Deploy (Local) por componentes
  shell: bash
  run: |
    set -e
    cd /ruta/al/proyecto
    git pull
    composer install --no-dev --optimize-autoloader
    php artisan optimize:clear
    php artisan migrate --force
    php artisan config:cache
    php artisan route:cache
    php artisan view:cache
    php artisan storage:link || true
    sudo systemctl reload php8.3-fpm
    sudo systemctl reload nginx
```

Script de despliegue

- Para remoto:

```
steps:
- name: Deploy (Remote) por SSH con Sail
  uses: appleboy/ssh-action@v1.0.0
  with:
    host: 203.0.113.10
    username: ubuntu
    password: MI_PASSWORD_SECRETA
    script: |
      set -e
      cd /ruta/al/proyecto
      git pull
      ./vendor/bin/sail down
      ./vendor/bin/sail up -d --build
      ./vendor/bin/sail artisan migrate --force
```

```
steps:
- name: Deploy (Remote) por SSH por componentes
  uses: appleboy/ssh-action@v1.0.0
  with:
    host: ${ secrets.SERVER_HOST }
    username: ${ secrets.SERVER_USER }
    password: ${ secrets.SERVER_PASSWORD }
    script: |
      set -e
      cd /ruta/al/proyecto
      git pull
      composer install --no-dev --optimize-autoloader
      php artisan optimize:clear
      php artisan migrate --force
      php artisan config:cache
      php artisan route:cache
      php artisan view:cache
      php artisan storage:link || true
      sudo systemctl reload php8.3-fpm
      sudo systemctl reload nginx
```

Script de despliegue

- Todos los ficheros anteriores están disponibles en Aules para su consulta.
 - La instrucción `set -e` hace que, si falla el despliegue, haga un rollback a su estado inicial.
 - Es una buena práctica no poner las credenciales en plano en este tipo de ficheros.
 - Una buena práctica sería utilizar Secrets que ofrece GitHub. Son como variables de entorno que se guardan en GitHub.
 - Lo tenéis de ejemplo en el script por componentes en remoto
 - <https://docs.github.com/en/actions/how-tos/write-workflows/choose-what-workflows-do/use-secrets>
-

Finalizando el CD

- Una vez tenemos nuestro fichero workflow finalizado, lo subiremos al repositorio con el flujo habitual (add, commit, push)
 - GitHub:
 - Detecta que hay un archivo en `.github/workflows/`
 - Lo registra como **workflow activo**
 - Aparece automáticamente en la pestaña **Actions**
 - Desde ese momento, GitHub queda “escuchando” eventos y cada push al main lanzará el despliegue automático.
 - Lo único que puede impedir este despliegue es que falle alguna condición técnica
-

Finalizando el CD

The screenshot shows the GitHub Actions interface for the repository 'JuanraCollado / biblioteca'. The 'Actions' tab is selected, displaying 'All workflows'. A sidebar on the left lists various management tools like Caches, Attestations, Runners, and Metrics. The main area shows a list of workflow runs. One run is visible: 'subido el workflow' for the 'main' branch, triggered by a push to commit 49615d4. The status is 'Queued' and it was triggered 1 minute ago.

Event	Status	Branch	Actor
subido el workflow	Queued	main	JuanraCollado

Juanra Collado

Finalizando el CD

← Deploy Laravel Sail

✓ probando el runner #3

Re-run all jobs ...

Summary

Jobs

✓ deploy

Run details

Usage

Workflow file

deploy

succeeded now in 11s

Search logs

Set up job

0s

```
1 Current runner version: '2.329.0'
2 Runner name: 'MacBook-Air-de-Juanra'
3 Runner group name: 'Default'
4 Machine name: 'MacBook-Air-de-Juanra'
5 ► GITHUB_TOKEN Permissions
9 Secret source: Actions
10 Prepare workflow directory
11 Prepare all required actions
12 Complete job name: deploy
```

> Deploy (Local) con Sail

6s

> Complete job

1s

Juanra Collado

Práctica 2 - Despliegue de Laravel en local

- En esta práctica aprenderemos a automatizar un despliegue de una aplicación Laravel en local utilizando el runner.
- Para ello, es imprescindible tener la práctica 1 realizada ya que esta crea el runner necesario para que el CD funcione en local.
- La práctica está desarrollada y resuelta en vídeo en Aules.

¿Dudas?

